

PainScope 3.0

Pain-First, Evidence-Grounded Problem Discovery Platform

“Topic 이 아니라, 구체적인 사용자 실패 경험의 변화를 찾는다.”

질문에 답할 수 있는 데이터를 먼저 판정하고,
대규모 사용자 의견에서 지금 조사해야 할 Pain Point 를 근거와 함께 발견한다.

MVP 기준

6 주 · 6 인 | CFPB Consumer Complaints = Main Product Proof | NHTSA Vehicle Complaints = Cross-domain Validation
기준일: 2026-08-19

V1.0 → V3.0 핵심 변화

항목	PainScope 1.0	PainScope 3.0
질문-데이터 연결	Domain 일치 여부 중심 Hard Routing	질문이 만들려는 주장에 필요한 증거를 해석하고 FULL / PARTIAL / UNSUPPORTED 로 Answerability 판정
Main / Validation Pack	Amazon Reviews 2023 = Main, CFPB = Secondary	CFPB Consumer Complaints = Main Product Proof, NHTSA Vehicle Complaints = Cross-domain Validation
AI 분석 단위	Raw review / complaint 전체를 embedding 후 cluster	문장·절 단위 Pain Candidate Claim 을 먼저 추출하고, 검증된 Failure cluster 만 Pain Point 로 승격
무엇을 찾는가	많이 언급되거나 baseline 에서 변하는 Topic / Cluster	구체적인 사용자 실패 경험 중 baseline 대비 Spike / Growing / Persistent 로 변하는 Pain
신뢰 장치	Cluster coherence, Evidence, limitation	Answerability + Pain Qualification + Evidence Traceability + 명시적 금지 해석
플랫폼 확장 증명	두 데이터셋에 동일 Pipeline 적용을 목표	Source Adapter + Data Pack Contract 만 바뀌서로 다른 CFPB/NHTSA schema 에서 동일 Core Engine 재사용

핵심 변화: 1.0 의 “관련 데이터에서 변하는 언급 묶음” → 3.0 의 “답변 가능한 Data Pack 에서 검증된 Pain Claim 의 변화”.

1. PROBLEM — 기획자는 Topic 이 아니라 “어디가 어떻게 실패하고 있는 지”가 필요하다

서비스 기획자·CX 담당자·신규사업 담당자는 “요즘 사용자가 무엇을 말하는가?”보다 “어떤 경험이 구체적으로 실패하고 있고, 그 실패가 최근 커지고 있는가?”를 알아야 다음 조사와 개선 기획으로 연결할 수 있다. 그러나 대규모 VOC 분석은 쉽게 Topic·Sentiment·빈도 집계로 수렴한다.

구분	예시	PainScope 3.0 처리
Topic	Account closure / Credit reporting / Steering	맥락·세그먼트로 사용하지만 Pain Point 로 노출하지 않음
Sentiment	화남 / 부정적 / 실망	보조 신호일 뿐 Pain 판정 기준이 아님
Pain Claim	계좌 폐쇄 요청이 반영되지 않음	분석 기본 단위
Impact	추가 수수료가 부과됨	Pain Frame 의 결과 필드
Cause	시스템 오류 때문에 발생	원문 근거가 없으면 자동 확정 금지
Solution	폐쇄 프로세스를 재설계해야 함	Discovery 결과가 아니라 후속 기획 단계

Pain Point 정의

사용자가 제품·서비스를 이용하는 특정 상황에서 기대한 상태·행동·결과가 충족되지 않아 발생한, 구체적으로 설명 가능한 장애·실패·불편. 제품/서비스의 어느 지점을 추가 조사해야 하는지 연결할 수 있어야 한다.

1.1 연구·데이터 관점 근거

- Complaint detection 연구에서는 complaint 를 단순 부정 감정이 아니라 “기대와 현실의 불일치/기대 위반”을 표현하는 행위로 다룬다. [R4]
- Painsight 연구 역시 keyword·aspect·topic 만으로는 구체적 pain point 를 충분히 분리하기 어렵다고 보고, sentiment/topic 정보를 결합해 불만 요인을 추출한다. [R5]
- PainScope 3.0 은 이를 제품화 관점으로 단순화해 “Pain Candidate Gate + Cluster-level Pain Frame Qualification”을 기본 설계로 채택한다.

2. COMPETITIVE LANDSCAPE — Theme/Trend 분석은 이미 강한 경쟁자가 많다

PainScope 는 경쟁사를 “기능이 부족한 제품”으로 전제하지 않는다. 오히려 2026 년 기준 Brandwatch, Qualtrics, Quid, Thematic 은 자연어 질의·자동 Topic/Theme·Emerging signal·근거 기반 분석을 빠르게 고도화했다. 따라서 3.0 은 Theme 생성 자체가 아니라 “질문-데이터 자격 + Pain Claim 자격 + 변화 근거”의 결합을 차별점으로 둔다.

서비스	강한 영역	관찰되는 한계/사용자 부담	PainScope 가 노리는 틈
Brandwatch	초대형 social/news coverage, Ask Iris 자연어 탐색, query/peak/summary	강력한 만큼 query/설정 학습비용이 있고 social source 특성상 질문에 필요한 모집단·거래·운영 데이터가 항상 존재하는 것은 아님 [C1][U1]	데이터가 질문의 어떤 주장까지 지지하는지 먼저 명시하고, 지원 불가 주장을 거절
Qualtrics	기업 내부 feedback 통합, Automated Topics, CX workflow	enterprise setup 과 advanced workflow 학습비용이 존재하고, 출발점이 보통 “이미 조직이 가진 feedback”임 [C2][U2]	데이터셋을 모르는 기획자의 질문에서 출발해 Data Pack 자격과 한계를 먼저 보여줌
Quid	2PB+ 외부 데이터, Dataset Library, Ask Q grounded answers	dataset 선택과 grounded Q&A 까지 PainScope 영역에 가장 가까움. 사용자 리뷰에서는 학습곡선이 언급됨 [C3][U3]	단순 dataset selection 이 아니라 supported_claims/required_dimensions 에 따른 FULL/PARTIAL/UNSUPPORTED contract

서비스	강한 영역	관찰되는 한계/사용자 부담	PainScope 가 노리는 틈
Thematic	bottom-up theme discovery, emerging issue, 원문 traceability, prioritization	Theme/Issue 탐지 자체는 매우 강해 PainScope 가 이 영역만으로 차별화 하기 어려움 [C4]	Raw Theme 를 곧바로 Pain 이라 부르지 않고 구체적 Failure/Impact 가 존재하는 cluster 만 Pain 으로 승격

2.1 우리 서비스의 장점 — 과장 없이 정의

- Answerability: “관련 있어 보이는 데이터”가 아니라 “이 질문의 어떤 주장까지 지지할 수 있는 데이터인가”를 계약으로 판정한다.
- Pain Qualification: Topic/Theme cluster 를 자동으로 Pain 이라고 부르지 않는다. Failure 가 구체적이고 Evidence 가 일치할 때만 Pain Point 로 승격한다.
- Change-first: 단순 Top Volume 보다 baseline 대비 증가·지속·신규성을 중심으로 조사 우선순위를 만든다.
- Explicit Limits: coverage, denominator, 국가/채널 편향, narrative availability 를 화면에 노출하고 금지된 해석을 함께 제공한다.
- Portability: 새 산업은 새 서비스를 만드는 대신 Source Adapter + Data Pack Contract 를 등록해 동일 Core Engine 을 재사용한다.

2.2 우리가 경쟁사보다 약한 점도 명시한다

- MVP source breadth 는 CFPB/NHTSA 2 개뿐이며 Brandwatch/Quid 처럼 광범위한 실시간 데이터 계약을 보유하지 않는다.
- 기업 내부 CRM·콜센터·설문 workflow 와의 통합은 Qualtrics 보다 훨씬 좁다.
- Thematic 처럼 장기간 운영된 theme governance·business outcome scoring 을 제공하지 않는다.
- 따라서 3.0 의 경쟁력은 “더 많은 데이터/더 좋은 대시보드”가 아니라 분석 자격과 Pain 자격을 명시적으로 통제하는 설계에 있다.

3. PRODUCT DEFINITION

3.1 Vision

Vision

다양한 산업의 대규모 사용자 경험 데이터를 Data Pack 으로 연결하고, 기획자가 데이터셋 이름이나 분석 기법을 몰라도 질문에서 출발해 “지금 조사할 가치가 커지는 구체적 Pain Point”를 발견할 수 있게 한다.

3.2 Primary Persona

- 서비스/프로덕트 기획자 — 신규 개선점 후보를 데이터로 좁히고 싶은 사람.
- CX/VoC 담당자 — 이미 알려진 Top Complaint 가 아니라 새롭게 커지는 세부 실패 경험을 보고 싶은 사람.
- 신규사업/전략 담당자 — 특정 산업의 공개 사용자 경험 데이터에서 조사 가설을 빠르게 만들고 싶은 사람.
- 데이터 분석 전문가는 Primary Persona 가 아니다. 사용자는 Spark, vector, clustering 을 몰라도 된다.

3.3 Product Promise

사용자가 원하는 것	PainScope 가 제공하는 것	제공하지 않는 것
내 질문을 어떤 데이터로 볼 수 있나?	Answerability + Data Pack + limitation	인터넷 전체 자동 수집
요즘 무엇이 문제인가?	검증된 Pain Cluster 의 변화 신호	단순 keyword top-N 만 제공
왜 지금 봐야 하나?	baseline/growth/persistence/support/spread	근거 없는 AI 우선순위
정말 이런 문제가 있나?	대표 Evidence + cluster/source lineage	LLM 생성문만 제시
어떻게 개선할까?	추가 조사 질문과 Discovery Brief	원인·해결책 자동 확정

3.4 END-TO-END USE CASE — 사용자는 데이터가 아니라 질문에서 시작한다

PainScope의 핵심 사용 흐름은 “데이터셋 선택 → 분석”이 아니다. 기획자는 먼저 알고 싶은 산업과 사용자 문제를 자연어로 입력한다. PainScope가 현재 등록된 Data Pack 중 해당 질문을 실제로 분석할 수 있는 Pack과 한계를 먼저 설명하고, 사용자가 범위를 선택하면 검증된 Pain Point 후보와 근거를 보여준다.

단계	사용자 경험
1. Ask	기획자가 “어떤 산업에서 어떤 불편이 커지고 있는가?”를 자연어로 입력한다.
2. Requirement	PainScope가 domain, target, question type, 필요한 text/time/segment/denominator를 구조화한다.
3. Answerability	등록된 Data Pack이 그 질문을 어디까지 지지할 수 있는지 FULL / PARTIAL / UNSUPPORTED로 판정한다.
4. Pack & Scope	사용자에게 분석 가능한 Pack, 기간, segment, limitation을 보여주고 사용자가 분석 범위를 선택한다.
5. Pain Discovery	선택 범위의 사전 계산된 Pain Claim/Cluster/Time Aggregate에서 검증된 Pain Point 후보를 조회·갱신한다.
6. Radar & Evidence	많은 Topic이 아니라 최근 비정상적으로 커지는 구체적 Failure를 Priority/Confidence/원문 Evidence와 함께 보여준다.
7. Action	기획자는 원문과 한계를 확인한 뒤 “추가 조사할 문제”를 선택하고 Discovery Brief를 만든다.

“분석 실행” 버튼의 실제 의미

사용자가 버튼을 누를 때마다 수백만 원문을 처음부터 LLM으로 다시 분석하지 않는다. Data Pack 별 Normalize → Pain Candidate → Embedding/Cluster → Pain Qualification → Time Aggregate는 배치·증분으로 미리 계산한다. 사용자의 Pack·기간·segment 선택은 이 결과 중 유효 범위를 정하고 Radar/Detail을 조회하는 요청이다. 새 데이터만 증분 처리해 결과를 갱신한다.

3.4.1 CFPB Main Use Case — 금융 서비스에서 “지금 조사할 실패 경험”을 찾는다

아래 예시는 PainScope의 Main Product Demo가 실제로 어떻게 보이는지를 설명한다. Pain Point 예시는 UI/분석 단위를 설명하기 위한 형태 예시이며, 최종 후보·증가율은 프로젝트에서 고정한 CFPB corpus를 실제 분석해 산출한다.

단계	사용자가 보는 것	PainScope 내부/의미
1. 탐색 요청	“요즘 신용카드 사용자에게 새롭게 커지는 불편은 뭐야?”	사용자는 CFPB라는 데이터셋 이름, Spark, clustering을 몰라도 된다.
2. 질문 구조화	화면에는 별도 기술정보를 강요하지 않음	domain=consumer_finance / target=credit_card / question_type=emerging_pain / required=text+time+segment
3. Data Pack 제안	CFPB Consumer Complaints 카드 노출	Product·Issue·Date·Consumer narrative를 가진 Pack이므로 complaint 내부에서 최근 커지는 Pain은 분석 가능. 단, 전체 카드 사용자 실제 발생률·원인 확정은 불가.
4. Answerability	“분석 가능하지만 데이터 한계가 있습니다”	공개 narrative cutoff와 자발적 complaint 편향을 표시. 현재시점 전체시장 prevalence 질문은 PARTIAL/UNSUPPORTED로 분리한다.
5. 범위 선택	Credit Card / 분석 가능 기간 / 필요 시 Issue 선택	선택 범위의 segment·coverage·minimum support를 다시 검사하고 실행 가능 범위를 확정한다.

단계	사용자가 보는 것	PainScope 내부/의미
6. Pain 후보 도출	예: “해지 요청 후에도 계정이 계속 활성 상태로 남음”, “분쟁 처리 후 동일 청구가 다시 나타남”	“Account closure”, “Billing dispute” 같은 Topic 이 아니라 문장·절에서 추출한 구체적 Failure claim 을 군집·검증한다.
7. 변화 탐지	Validated Pain 만 Radar 에 표시	각 Pain 의 conditional share 를 baseline 과 비교해 Spike / Growing / Persistent 를 판정하고 support·spread·persistence 를 함께 보여준다.
8. 근거 확인	Pain 을 클릭해 대표 complaint 원문과 추세를 확인	Context / Failure / Impact / Target, 원문 Evidence, company 집중도, data limitation 을 한 화면에서 보여준다.
9. 기획 Action	“추가 조사 대상으로 저장”	원인·해결책을 자동 확정하지 않고 발견 내용·근거·한계·다음 사용자 조사 질문으로 Discovery Brief 를 생성한다.

CFPB 에서 PainScope 가 보여주려는 차이

“Credit reporting 이 많이 언급된다”가 결과가 아니다. 같은 Product/Issue 안에서도 “정정된 정보가 다시 나타남”, “해지 요청이 반영되지 않음”처럼 무엇이 어떻게 실패했는지까지 내려간 Claim 만 Pain 후보가 된다. 그중 최근 baseline 에서 이례적으로 커지는 후보가 Radar 상단에 올라온다.

3.4.2 NHTSA Validation Use Case — 같은 엔진이 다른 산업에서도 동작하는가

NHTSA 는 자동차 Safety 전용 제품을 만들기 위한 두 번째 서비스가 아니다. CFPB 와 전혀 다른 schema 에서도 Data Pack Contract 와 Source Adapter 만 바꾸면 동일 Pain Discovery Core 가 재사용되는지 보여주는 Cross-domain Validation 이다.

단계	사용자가 보는 것	PainScope 내부/의미
1. 탐색 요청	“최근 자동차 이용자들이 조향 장치에서 새롭게 겪는 불편은 뭐야?”	domain=automotive / intent=emerging_pain
2. Data Pack 제안	NHTSA Vehicle Complaints	Vehicle/Component/Narrative/Date/Crash·Fire·Injury 등의 필드로 complaint 기반 Pain 변화 분석 가능
3. 범위 선택	Component=STEERING, 기간 또는 차량 범위 선택	CFPB 의 Product/Issue 대신 Component/Make/Model 을 segment 로 사용
4. Pain 후보	예: “주행 중 power steering assist 가 갑자기 사라짐”, “시동 직후 steering 이 잠겨 움직이지 않음”	“Steering” Topic 이 아니라 실제 Failure claim 을 동일 Candidate → Cluster → Qualification 흐름으로 처리
5. Radar/Detail	동일한 Spike/Growing/Persistent, Priority, Confidence, Evidence UI	도메인별 severity mapping 만 바꿨고 Core Change/Evidence Engine 은 재사용
6. 해석 한계	“complaint 에서 이 Failure 의 비중이 증가”로 표현	실제 차량 전체 고장률·제조 결함·원인은 complaint 만으로 확정하지 않음
공통 개념	CFPB Pack	NHTSA Pack
분석 도메인	Consumer Finance	Automotive
Coarse Segment	Product / Sub-product / Issue	Component / Make / Model
사용자 원문	Consumer narrative	Complaint narrative
시간	Date received	Complaint / incident time
Severity 보조	Issue·metadata 기반	Crash / Fire / Injury 등
Core 표현	event_time + text + segment + severity + metadata	event_time + text + segment + severity + metadata

Same Engine, Different Data Pack

CFPB에서 Product/Issue를 사용하던 부분을 NHTSA에서는 Vehicle/Component로 매핑한다. 그 뒤의 Pain Candidate Gate, claim embedding, Failure clustering, Pain Qualification, time aggregation, change detection, Evidence trace는 동일 Core Engine을 재사용한다. 이 장면이 “PainScope는 금융 민원 분석기가 아니라 Data Pack 기반 Problem Discovery Platform”이라는 주장을 증명한다.

3.4.3 현재 지원하지 않는 질문은 어떻게 보이는가

예: “지금 사람들이 의료서비스에 대해 느끼는 불편함을 알고 싶어.”

MVP에 등록된 실제 Pack은 CFPB와 NHTSA 뿐이므로 healthcare 질문은 UNSUPPORTED다. PainScope는 가장 가까워 보이는 CFPB/NHTSA를 억지로 추천하지 않고 “현재 분석 가능한 의료 Data Pack이 없습니다”라고 종료한다. 향후 의료당국 complaint처럼 text + time + segment를 갖는 데이터가 Data Pack Contract로 등록되면 같은 Ask → Answerability → Scope → Pain Discovery 흐름을 그대로 재사용하는 것이 최종 플랫폼 비전이다.

이 구조에서 “다양한 산업에 적용 가능”은 모든 산업을 현재 지원한다는 뜻이 아니다. 새로운 산업마다 별도 제품을 다시 만드는 대신, 해당 산업의 사용자 경험 데이터를 Data Pack으로 계약화하고 동일 Core Engine에 연결하는 것이 확장 단위다.

4. ANSWERABILITY ENGINE — 질문과 데이터의 “자격”을 판정한다

Answerability는 Domain Router가 아니다. 사용자가 만들고 싶은 주장에 어떤 증거가 필요한지 구조화하고, Data Pack이 그 증거를 실제로 가지고 있는지 deterministic rule로 판정한다.

Question Type	필요 차원	예시 판정
Emerging Pain	text + time + segment	CFPB/NHTSA: FULL 또는 coverage에 따라 PARTIAL
Comparison	text + comparable segment + period	두 segment taxonomy가 비교 가능할 때 FULL
Prevalence	population/transaction denominator	CFPB complaint만으로 UNSUPPORTED
Cause	causal/operational evidence	Complaint narrative만으로 PARTIAL/UNSUPPORTED
Future Prediction	stable target + label + forecast contract	MVP 기본 UNSUPPORTED

4.1 판정 등급

등급	의미	화면 동작
FULL	질문의 핵심 주장에 필요한 필드와 coverage가 존재	분석 범위와 limitation을 보여주고 실행
PARTIAL	일부 주장은 가능하지만 모집단·원인·coverage 등 핵심 제약 존재	가능/불가능 주장을 분리해 설명 후 실행 선택
UNSUPPORTED	필수 차원이 없거나 Data Pack이 지원하지 않음	가장 가까운 데이터로 억지 분석하지 않고 거절

4.2 구현 순서

1. Question Requirement Extractor: 자연어에서 domain, target, question_type, required_dimensions를 JSON schema로 추출한다.
2. Contract Evaluator: LLM 판단이 아니라 Data Pack metadata와 hard rule이 최종 FULL/PARTIAL/UNSUPPORTED를 결정한다.

3. Scope Re-check: Product/기간/segment 를 고른 뒤 실제 row count, text coverage, minimum support 를 반영해 판정을 갱신한다.
4. Limitation Composer: 실패한 조건을 “이 데이터로 말할 수 있는 것 / 말할 수 없는 것”으로 사용자에게 번역한다.
5. Audit Log: requirement, selected pack, failed rule, contract_version 을 저장해 결과를 재현한다.

5. DATA PACK 3.0 — 파일이 아니라 분석 계약 + Pain Extraction 계약

3.0 에서는 기존 Data Pack metadata 에 “이 데이터에서 Pain Candidate 를 어떻게 만들 것인가”를 추가한다. Source-specific 지식은 Pack 에 두고, Core Pain Discovery Pipeline 은 최대한 domain-independent 하게 유지한다.

필드	역할	예시
supported_domains / journeys	질문 도메인 · 사용 경험 범위	consumer_finance / financial_service_experience
supported_claims	허용 가능한 분석 주장	emerging_pain, comparison
available_fields	실제 분석 가능 필드	text, time, product, issue
segment_path	coarse→fine segmentation	Product > Sub-product > Issue
weak_label_fields	후보 추출 보조	Issue, Sub-issue / Component
pain_candidate_policy	문장/절 추출 · 필터 규칙	failure cues + semantic prototype
severity_mapping	도메인별 심각도 보조	CFPB metadata / NHTSA crash-fire-injury
taxonomy_version	시간에 따른 schema 변화 통제	CFPB form taxonomy version
coverage / known_bias	데이터 기간 · 대표성 한계	자발적 complaint, 국가 편향
status / version	현재 실행 가능성	historical_text_ready / fresh_text_ready

5.1 CFPB — Main Product Proof

- 역할: 금융 서비스에서 실제 사용자 complaint 의 세부 Pain 을 찾고, 대규모 시간 변화 분석과 제품 가치를 증명하는 Main Data Pack.
- 핵심 구조: Product / Sub-product / Issue / Sub-issue / Date received / Company / Consumer narrative(과거 공개분).
- Issue/Sub-issue 는 Pain 정답이 아니라 coarse segmentation 과 weak label 로 사용한다. 같은 Issue 안에서 narrative 의 더 구체적인 Failure 를 다시 찾는다.
- 중요한 2026 정책 변화: CFPB 는 2026-08-14 부터 complaint narratives 의 재량적 공개를 중단했고, 기존 공개 narrative 는 FOIA 목적상 public domain 으로 보고 Reading Room 을 통해 공개한다고 밝혔다. [D1][D2]
- 따라서 MVP Main Demo 는 “historical narrative corpus”를 명시적으로 고정한다. current market realtime 이라고 주장하지 않는다.
- 신규 structured complaint 는 별도 freshness signal 로 사용할 수 있지만, narrative 기반 신규 Pain 생성은 확보한 historical text cutoff 범위에서 수행한다.

5.2 NHTSA — Cross-domain / Fresh Text Validation

- 역할: 자동차 전문 제품을 만드는 것이 아니라, CFPB 와 다른 schema 에서도 같은 Pain Discovery Core 가 동작함을 증명한다.
- NHTSA complaint data 는 vehicle make/model/model year, component, complaint narrative, crash/fire/injury 등의 구조를 제공하고 defect trend 탐지에 활용된다. [D3]
- CFPB 의 Product/Issue 체계 대신 Vehicle/Component 를 segment_path 와 weak_label_fields 로 사용한다.
- MVP UI 에서 Recall/Investigation prediction 까지 확장하지 않는다. NHTSA 는 Same Engine, Different Data Pack 을 보여주는 30~60 초 검증 장면이다.

6. PAIN-FIRST AI PIPELINE — 3.0 의 핵심 기술 변경

핵심 변경

Raw Narrative → Embedding → Cluster 를 기본 경로에서 제거한다. 대신 Raw Narrative → Pain Candidate Claim → Segment-aware Embedding → Pain Cluster → Pain Frame Qualification → Temporal Discovery 를 사용한다.

1 Normalize source schema → common event	2 Clause Split 긴 narrative 를 문장·절로 분할	3 Candidate Gate 실패 경험 후보만 남김	4 Segment source taxonomy 로 분석공간 축소
5 Embed candidate claim 만 vector 화	6 Bisect 세부 failure cluster 생성	7 Qualify Pain Frame / invalid / mixed	8 Detect validated Pain 만 시간 변화 추적

6.1 Job 1 — Normalize

- 원천별 Source Adapter 가 event_id, event_time, text, segment metadata, entity, severity_signal 을 Common Event 로 변환한다.
- CFPB taxonomy 변화는 canonical mapping table 로 보정하고 원래 label 과 canonical label 을 모두 보존한다.
- bad record, null text, timestamp 이상, duplicate key 를 quarantine 한다.

6.2 Job 2 — Sentence / Clause Split

- 긴 complaint 하나에는 배경·감정·여러 실패 경험이 함께 존재할 수 있으므로 event 전체를 embedding 하지 않는다.
- 문장 단위 분리 후 접속사/구두점 기반으로 너무 긴 문장은 clause 수준으로 한 번 더 나눈다.
- 한 complaint 에서 최대 2~3 개의 Pain Candidate 를 허용해 한 사건이 여러 개선점을 포함할 수 있게 한다.
- event_id → claim_id 의 1:N 관계를 유지하여 원문 Evidence 로 역추적한다.

6.3 Job 3 — Pain Candidate Gate

모든 문장을 embedding 하면 다시 Topic 분석으로 돌아간다. 따라서 먼저 “구체적인 실패/장애/원치 않은 결과를 기술하는 문장인가?”를 빠르게 판정한다. 이 단계는 대규모 행렬 LLM 호출을 사용하지 않는다.

신호	역할	예시
Failure cues	실패·불능·거절·지연·누락·오류 표현	failed, unable, denied, still open, missing, delayed
Outcome cues	사용자에게 생긴 부정적 결과	fee charged, money lost, crash, injury
Negation/expectation	기대 상태가 충족되지 않음	requested X but Y, should have, never received
Weak label context	source taxonomy 로 오탐 축소	CFPB Issue/Sub-issue, NHTSA Component
Semantic prototype	표현이 달라도 failure 의미가 가까운 문장 포착	“request ignored”, “service unavailable”, “unexpected charge”

Gate 정책: 규칙 점수 + semantic prototype similarity 를 합쳐 후보를 선택하되, negative sentiment 만 있는 문장은 통과시키지 않는다. 단순 “I am angry”보다 “I requested closure three times but the account remained open”이 우선된다.

6.4 Job 4 — Segment-aware Embedding

- CFPB 전체를 한 번에 군집화하지 않는다. Product → Sub-product/Issue 단위로 먼저 분석 공간을 제한한다.
- NHTSA 는 Component 와 필요 시 Make/Model 을 이용해 coarse segment 를 만든다.
- Embedding 대상은 full narrative 가 아니라 Pain Candidate claim 이다. 영어 텍스트 기준 BGE-small-en-v1.5 계열 384-d sentence embedding 을 기본안으로 둔다.
- 모델이 배포되지 않거나 처리량이 부족하면 TF-IDF baseline 으로 전환하되 vector space 를 섞지 않는다.

6.5 Job 5 — BisectingKMeans 로 세부 Failure Cluster 생성

- 일반 K-Means 의 고정 K 보다 coarse cluster 를 단계적으로 나눌 수 있는 Spark MLlib BisectingKMeans 를 기본안으로 둔다.
- segment 내부 candidate 수가 매우 작으면 cluster 를 만들지 않고 “insufficient support”로 보류한다.
- minimum cluster size 를 두어 1~2 건의 우연한 표현을 Pain 으로 승격하지 않는다.
- cluster 는 시간과 독립적으로 historical corpus 전체에서 생성하고, 이후 event_time 을 이용해 기간별 분포만 계산한다.

6.6 Job 6 — Cluster-level Pain Frame Qualification

LLM 은 여기서 처음 핵심적으로 사용한다. 단, row 단위가 아니라 cluster 당 한 번이다. centroid 근접도·중복 제거·시간 다양성을 고려한 대표 Evidence 12~20 개만 입력하고 구조화 JSON 을 받는다.

출력 필드	의미
valid_pain	구체적인 실패 경험으로 해석 가능한가
context	어느 상황/과정에서 발생했는가
failure	무엇이 기대대로 작동하지 않았는가
impact	사용자에게 어떤 결과가 생겼는가; 근거 없으면 null
target	추가 조사할 제품/서비스 과정
canonical_statement	근거를 벗어나지 않은 1 문장 Pain 표현
mixed_cluster	대표 Evidence 가 둘 이상의 실패를 섞고 있는가
unsupported_inference	원인·법 위반 등 근거 밖 해석이 포함되는지 점검

판정 규칙: valid_pain=false 이면 Radar 후보에서 제거한다. mixed_cluster=true 이고 support 가 충분하면 해당 cluster 만 한 번 추가 bisect 한다. 재분할 후에도 혼합이면 “Topic Candidate”로 남기고 Pain Point 로 노출하지 않는다.

왜 row-level Pain Frame 을 만들지 않는가

수백만 complaint 에 생성형 LLM 을 호출하면 비용·시간·재현성이 무너진다. 3.0 은 값싼 분산처리로 후보를 축소하고, cluster 당 제한된 대표 Evidence 에서만 LLM 을 사용해 6 주 프로젝트 안에서 품질과 규모를 동시에 잡는다.

6.7 Job 7 — Pain ID / Assignment Radius / Novel Queue

- 검증된 Pain Cluster 마다 stable pain_id, centroid, model_version, contract_version, canonical Pain Frame 을 저장한다.
- 신규 claim 은 저장 centroid 에 가장 가까운 Pain 으로 배정하되, cluster 별 기존 member 의 centroid distance 분포에서 자동 계산한 assignment radius 를 넘으면 억지 배정하지 않는다.
- radius 밖 claim 은 Novel Candidate queue 에 축적하고 support 가 일정 수준을 넘을 때만 신규 cluster job 대상으로 올린다.
- 이 방식은 “새 표현을 기존 Topic 에 강제로 넣는 문제”를 줄이고 Novel Pain 을 별도 경로로 관리한다.

7. TEMPORAL PAIN DISCOVERY — Cluster 크기가 아니라 변화의 의미를 본다

검증된 Pain Point 만 시간 분석에 들어간다. Cluster Size 는 “Pain 이 맞는가”를 결정하는 기준이 아니라 Evidence support 일 뿐이다.

Feature	계산	해석
Volume	segment × pain × period count	근거량
Conditional Share	pain count / 같은 segment 의 전체 valid pain claims	기간별 분모를 고려한 상대 비중
Baseline	rolling median + MAD 또는 EWMA	평상시 수준
Anomaly	최근 share 가 baseline 에서 벗어난 정도	갑작스러운 변화
Growth	최근 window 와 기준 window 의 증가율/기울기	커지는 속도
Persistence	threshold 초과 연속 기간	일시적 spike 와 지속 이슈 구분
Severity	Data Pack 별 구조화 신호	사용자 영향 보조
Spread	company/entity 집중도	단일 entity 사건인지 폭넓은 문제인지 구분

7.1 Discovery State

State	의미	기본 판정
Spike	짧은 기간 급증	Robust anomaly 가 높지만 지속 기간이 짧음
Growing	여러 기간 연속 증가	최근 window 의 증가 + baseline 상회
Persistent	오랜 기간 기준선 위	연속 threshold 초과
Novel	기존 pain radius 밖 신규 cluster 가 support 확보	Novel queue → 신규 Pain Qualification

7.2 Discovery Priority

사업기회 점수가 아니라 “추가 조사 우선순위”다. 학습된 정답 모델이 아니라 설명 가능한 휴리스틱으로 유지한다.

요소	기본 비중	이유
Anomaly	30%	평소와 얼마나 다른가
Growth	25%	얼마나 빠르게 커지는가
Persistence	15%	일시적 noise 인가
Support	15%	판단할 근거가 있는가
Severity	10%	영향이 큰가
Spread	5%	특정 entity 하나에만 쏠렸는가

7.3 Confidence 는 별도

- Pain Qualification: 대표 Evidence 가 같은 Failure 를 설명하는가.
- Sample Sufficiency: minimum support 와 기간별 분모가 충분한가.
- Evidence Diversity: 중복 문장·단일 company/entity 쓸림이 심하지 않은가.
- Data Completeness: 선택 기간의 narrative/metadata coverage 가 안정적인가.
- Answerability 는 query-level 자격이므로 Confidence 에 섞지 않고 화면 상단에 FULL/PARTIAL/UNSUPPORTED badge 로 유지한다.

7.4 절대로 주장하지 않을 것

- “이 문제를 겪는 전체 소비자가 300% 증가했다” — 모집단 denominator 가 없으면 금지.
- “이 회사가 원인이다” — complaint evidence 만으로 인과 확정 금지.
- “이 Pain 이 가장 중요한 개선과제다” — Discovery Priority 는 조사 우선순위일 뿐 최종 사업 우선순위가 아님.

8. DISTRIBUTED SYSTEM DESIGN — “Spark 를 쓴다”가 아니라 분산 처리 자체를 증명한다

PainScope 3.0 의 분산 설계는 단순히 대용량 파일을 Spark 로 읽는 데 목적이 있지 않다. 전체 complaint 를 기간·segment 의 분모로 유지한 채 문장/절 Claim 으로 확장하고, Pain Candidate 추출·embedding·분산 clustering·pain×segment×time 집계·중분 갱신을 반복한다. 특히 Claim 확장 이후의 vector workload 와 cluster×time GroupBy 가 계산량과 shuffle 을 크게 만들기 때문에, 파티션·Worker·Join·Incremental 전략이 실제 결과 생성 비용을 좌우한다. [S1][S2]

분산 프로젝트의 검증 원칙

“단일 노드에서는 절대 불가능하다”고 과장하지 않는다. 동일한 논리 결과를 만드는 workload 를 1/2/4 Worker 에서 실행하고, 어느 규모부터 Scale-Out 이 유효해지는지와 scan·shuffle·spill·skew 병목을 수치로 설명한다. Spark 사용 여부가 아니라 “분산 전략 이 처리 비용을 어떻게 바꿨는가”가 발표의 증거다.

8.1 왜 PainScope 에 분산 처리가 필요한가

Workload	분산이 필요한 이유	분산 방식	핵심 지표
전체 분모 유지	Emerging Pain 은 절대 건수가 작아도 특정 segment 내 점유율이 빠르게 증가 할 수 있다. 샘플링하면 희귀 신호와 기간 별 denominator 가 함께 왜곡될 수 있다.	기간·source 파티션 전체 scan + 분산 aggregate	scan bytes / records·sec
Claim row 확장	complaint 1 건이 여러 sentence/clause 로 explode 되며 원본 event 보다 처리 row 가 증가한다.	Spark explode/filter 를 partition 단 위로 병렬 실행	input→claim row 수 / task throughput
Pain Candidate / Embedding	모든 claim 을 embedding 하지 않고 Gate 로 줄이지만, 후보 집합에 대한 vector inference 는 여전히 CPU-heavy 다.	mapPartitions 기반 batch inference, executor 별 모델 1 회 load	candidate ratio / claims·sec / executor CPU
Failure Clustering	segment 내부의 수많은 vector 를 반복 분할해 세부 Failure cluster 를 만든다.	Spark MLlib BisectingKMeans distributed iteration [S3]	fit wall time / stage time
Pain×Time 집계	pain_id×segment×period 의 count/share 와 denominator 를 만들 때 대규모 groupBy·join·window 가 발생한다.	repartition + aggregate + AQE/skew 제어 [S2]	shuffle read/write / spill / skew
중분 갱신	새 complaint 가 들어올 때 전체 corpus 를 다시 cluster 하면 비용이 크다.	Structured Streaming/file micro-batch + centroid broadcast + idempotent upsert [S4]	update latency / reprocessed rows

8.2 실행 토폴로지 — 최소 2 개 Worker 노드에서 실제 분산 실행

MVP 는 Spark Standalone 을 사용한다. Spark 공식 Standalone 모드는 Master 와 여러 Worker 로 cluster 를 구성할 수 있고, 동일 프로그램에서 Worker 수를 통제해 Scale-Out 실험하기 쉽다. [S1] 개발 편의상 단일 PC 의 multi-container 를 사용할 수는 있지만, 최종 분산 증명은 가능하면 별도 VM/호스트의 Worker 2 개 이상에서 측정한다.

계층	구성	역할
Source / Data Pack	CFPB historical snapshot, NHTSA complaint snapshot, 신규 file/Kafka replay	원천은 immutable snapshot 으로 보존
Object Lake	MinIO + Parquet: Raw / Clean / Claim / Candidate / Feature / Aggregate	Spark Worker 가 동일 object path 를 병렬 scan
Spark Control	1 Master + Pipeline Driver	Driver 는 DAG 제출·stage orchestration, Master 는 Worker resource 할당

계층	구성	역할
Spark Compute	Worker 1~4 / 각 Worker 에 Executor	Normalize, Candidate, embedding, clustering, aggregate 를 partition 단위 실행
Serving	PostgreSQL + API	사용자 요청은 Spark 전체 Job 을 직접 실행하지 않고 사전 계산 결과를 조회
Observability	Spark UI + Event Log + History Server	Job/Stage/Task, executor, shuffle, spill, skew 를 기록 [S5]

권장 실습 구성

1 Master / 2~4 Worker / 1 MinIO / 1 PostgreSQL / 1 API 를 Docker Compose 또는 VM 으로 분리한다. 1 Worker 와 4 Worker 를 동일 데이터·동일 코드로 비교할 수 있어야 한다. Kafka 는 필수가 아니라 5 주차 증분 입력 source 의 선택지다.

8.3 데이터 레이크 계층과 파티션 계약

Layer	저장 내용	물리/논리 Partition	읽기·수명
Raw	원천 CSV/JSON 과 ingest metadata	source_id / ingest_date	재처리·schema 변경의 기준. 수정하지 않는 immutable layer
Clean	Common Event: event_id, event_time, text, segment, severity, metadata	source_id / event_year_month	기간·Data Pack 기준으로 반복 scan
Claim	sentence/clause 단위 claim_id + parent event_id	source_id / event_year_month	explode 결과. physical partition 을 과도하게 늘리지 않음
Candidate	Pain Gate 를 통과한 claim + gate score	source_id / event_year_month	embedding 대상만 축소
Feature	claim_id + vector + model_version	domain_id / model_version + date bucket	cluster 재실행·centroid assignment 에 재사용
Pain Model	pain_id, centroid, radius, frame, cluster_version	versioned table	실시간/증분에서 broadcast 가능한 소형 model artifact
Aggregate	pain×segment×period count/share/baseline	domain_id / granularity / period	Radar·Detail API 가 읽는 핵심 시계열
Evidence	pain_id 별 대표 claim/event pointer	domain_id / pipeline_version	원문 trace 용. 전체 원문을 PostgreSQL 에 복제하지 않음

- 물리 Parquet partition 은 source/date 처럼 반복 필터가 걸리고 cardinality 가 낮은 축에만 둔다. product_id, pain_id, company 처럼 값이 많은 키를 directory partition 으로 쓰지 않아 small-file 폭증을 막는다.
- segment_path 는 저장 partition 이 아니라 Spark 내부 repartition key 또는 segment_bucket(hash 기반)으로 사용한다.
- 파일이 지나치게 작아지면 batch 종료 시 compaction 을 수행하고, 기간 필터에서는 Parquet column pruning/predicate pushdown 이 동작하도록 필요한 column 만 읽는다.
- 모든 산출물은 data_pack_version / code_version / model_version / cluster_version / baseline_version 을 함께 저장해서 다른 vector·cluster 공간을 섞지 않는다.

8.4 Distributed Job DAG — Narrow 처리와 Wide/Shuffle 지점을 명확히 구분

Job	입력→출력	주요 연산	Shuffle 성격	분산 설계
D1 Normalize	Raw → Clean Event	select/filter/withColumn, schema map, event_id, dedupe	대부분 Narrow; global dedupe 는 Shuffle 가능	source/date parallel scan
D2 Split	Event → Claim	sentence/clause split + explode	Narrow 지만 row 수 확대	partition-local expansion
D3 Candidate	Claim → Pain Candidate	rule/lexicon/prototype filter, small metadata join	Narrow / Broadcast	비싼 feature 이전에 corpus 축소
D4 Feature	Candidate → Vector	BGE batch embedding 또는 TF-IDF	Narrow, CPU-heavy	mapPartitions, executor batch inference

Job	입력→출력	주요 연산	Shuffle 성격	분산 설계
D5 Cluster	Vector → Candidate Cluster	segment-aware BisectingKMeans	Wide / iterative shuffle	MLlib distributed fit
D6 Assign & Rep	Vector → pain assignment / representative	centroid transform, cluster 별 representative Top-K	Top-K 에서 Shuffle	Feature 재사용; representative 만 다음 단계로
D7 Qualify	Cluster → Validated Pain	대표 Evidence 10~20 개 + limited LLM	Spark 밖의 제한 호출	driver 가 전체 row 를 collect 하 지 않음
D8 Aggregate	Pain → pain × segment × period	count, denominator, share, entity spread	가장 큰 Wide/Shuffle	repartition/groupBy/join
D9 Detect	Aggregate → Signal	baseline, growth, persistence, state	pre-aggregate 후 Window	pain_id 별 partition-local time order
D10 Evidence	Signal → Evidence Index	대표 원문 · lineage Top-K	작아진 결과에서 Window	PostgreSQL serving 용 metadata 만 적재

가장 큰 병목을 어디로 예상하는가

D4 Feature 는 CPU 추론 병목, D5 Cluster 는 iterative compute, D8 Aggregate 는 network shuffle 병목이다. 최적화 우선순위
도 이 세 단계에 둔다. UI 나 LLM 호출이 아니라 Spark Stage 의 scan/shuffle/spill 을 먼저 관측한다.

8.5 Partition / Join / Skew 전략

문제	기본 전략	설계 의도
Scan	source_id + event_year_month partition pruning	분석 기간과 Data Pack 을 먼저 줄인 뒤 필요한 column 만 read
Repartition	segment_path 자체 대신 segment_bucket 또는 분석 대상 segment key	Cluster/Aggregate 전에 동일 key 의 row 를 같은 partition 근처로 모음
Broadcast Join	Data Pack metadata, Product/Component mapping, Pain centroid 처럼 작은 테이블	대규모 fact table shuffle 을 피함
Large Join	claim ↔ feature ↔ assignment 처럼 큰 테이블	join key 를 동일하게 repartition 하고 사용 column 을 사전 prune
AQE	runtime statistics 기반 partition coalesce / skew 대응을 활성화 [S2]	자동 최적화를 활용하되 전/후 shuffle read/write 를 비교
Skew Control	특정 Product/Issue 가 한 task 를 오래 점유하면 salting 또는 segment split	p95 task time 과 median 의 격차, spill, straggler task 를 관측
Cache/Persist	Candidate Feature 처럼 cluster/assignment 에 서 반복 사용되는 중간 결과만 persist	Raw/Clean 전체는 무조건 cache 하지 않아 executor memory 압박 방지
Shuffle Partitions	고정 default 를 맹신하지 않고 executor core · 입 력 크기에 맞춰 조정	너무 적으면 skew/대형 partition, 너무 많으면 scheduler/small-task overhead

8.6 Embedding 을 실제로 분산하는 방법

1. Candidate Gate 를 먼저 수행한다. 모든 raw complaint 를 vector 화하지 않고 Pain 가능성이 있는 claim 만 Feature Job 으로 넘긴다.
2. Feature DataFrame 을 적정 partition 수로 repartition 한 뒤 mapPartitions 계열 처리를 사용한다. executor 가 partition 처리 시작 시 BGE 모델을 한 번만 load 하고 claim 을 batch 로 묶어 추론한다. row 마다 모델을 로드하는 Python UDF 는 금지한다.
3. 출력은 claim_id, vector, model_version, quality_flag 로 Parquet 에 저장한다. 같은 version 의 Feature 는 clustering, centroid 계산, Evidence representative 선정에서 재사용한다.
4. 처리량이 일정 내 확보되지 않으면 Candidate Gate 를 강화하고, 최후 fallback 으로 Spark TF-IDF 를 사용한다. fallback 시 batch/증분 모두 동일 vector space 로 맞춰 서로 다른 feature 를 혼용하지 않는다.

분산의 핵심

embedding 모델 자체가 “분산 모델”인 것이 아니다. 큰 Candidate DataFrame 을 여러 executor partition 으로 나누고 각 executor 가 batch inference 를 수행하도록 만들어 vector workload 를 수평 확장하는 것이 핵심이다.

8.7 Segment-aware Distributed Clustering

PainScope 는 전체 금융 complaint 를 한 번에 cluster 하지 않는다. Product/Issue 또는 Component 같은 coarse segment 로 먼저 범위를 좁힌 뒤 그 안에서 세부 Failure 를 찾는다. 대형 segment 는 별도 Spark Job 으로 분리하고, 작은 segment 는 동일 실행에서 순차/병렬 schedule 한다. BisectingKMeans 는 같은 level 의 분할을 함께 실행해 parallelism 을 높이는 Spark MLlib 구현을 사용한다. [S3]

단계	구현	의도
입력	Validated Candidate Feature + segment_path	대분류가 cluster 를 지배하지 않도록 coarse segment 별 semantic space 를 분리
Fit	Spark MLlib BisectingKMeans	분산 iteration 으로 세부 Failure 후보 생성
Model Artifact	centroid + assignment radius + cluster_version	중분 event 에서 기존 Pain 에 빠르게 assign 하기 위한 broadcast artifact
Representative	centroid 근접도 + 시간/회사 다양성을 고려한 Top-K	LLM 에는 cluster 전체가 아니라 대표 Evidence 만 전달
Mixed Cluster	Quality Gate 실패 시 1 회 bisect 재분할	재실패하면 Pain 으로 승격하지 않고 숨김
Novel	centroid 최대 유사도가 radius 밖이면 Novel Queue	실시간 재-cluster 대신 다음 batch rebuild 후보로 누적

8.8 Aggregate & Detect — 네트워크 Shuffle 이 가장 큰 구간

Validated Pain 이 만들어진 뒤부터는 텍스트 모델보다 분산 집계가 핵심이다. 단순 pain count 가 아니라 같은 segment·period 의 전체 complaint/claim 을 분모로 conditional share 를 계산하고, 이 시계열에서 baseline·growth·persistence 를 판정한다.

Feature	계산	Spark 포인트
pain_count	segment×pain_id×period count	groupBy / shuffle aggregate
denominator	segment×period 전체 유효 event/claim count	별도 aggregate 후 join 또는 window
conditional_share	pain_count / denominator	분자·분모 join; 동일 partition key 정렬
entity_spread	unique company/make 등 분산 정도	approx distinct / Top-1 concentration
baseline	과거 rolling 기준선	pre-aggregated time series 에 Window 적용
growth / persistence	최근 변화와 연속 초과 기간	pain_id 별 time-order Window
priority / confidence	Discovery Priority + quality/coverage	작어진 aggregate table 에서 계산

- D8 Aggregate 전에 key 를 domain_id/segment_bucket/period 기준으로 repartition 해 같은 key 의 shuffle 을 예측 가능하게 만든다.
- 회사 하나에 매우 큰 데이터가 몰리면 company 자체를 partition key 로 고정하지 않고 salting/2 단계 aggregate 로 straggler 를 줄인다.
- Detect 는 원문 수백만 row 가 아니라 이미 축소된 pain×period 시계열에서 수행하므로 Window 비용과 제품 응답 시간을 분리한다.

8.9 Batch 와 Serving 의 경계 — 사용자 질문이 Spark Full Job 을 직접 실행하지 않는다

경계	처리	왜 이렇게 나누는가
Offline Batch	Normalize→Claim→Candidate→Feature→Cluster→Qualify→Aggregate→Detect→Evidence	고비용 작업. Data Pack/version 단위로 사전 계산
Publish	검증 완료된 pipeline_version 만 active_version 으로 전환	부분 실패 산출물이 API 에 노출되지 않도록 immutable output + version publish
Online API	Ask/Answerability/Scope/Radar/Detail/Evidence	PostgreSQL aggregate/profile/evidence metadata 조회

경계	처리	왜 이렇게 나누는가
New Data	증분 assignment/aggregate	전체 re-cluster 없이 기존 pain centroid 에 배정하고 Radar 시계열만 갱신

UX 성능 원칙

“분석 시작”은 매번 수백만 문장에 Spark Job 을 새로 거는 버튼이 아니다. 기획자가 Pack/Scope 를 확정하면 사전·증분 계산된 결과를 범위에 맞게 조회한다. 이렇게 해야 대규모 분산 계산과 사용자 인터랙션 latency 를 분리할 수 있다.

8.10 Incremental Pipeline — Re-clustering 없이 새 Pain 신호를 갱신

단계	방식	핵심 보장
1 Ingest	새 file 또는 Kafka micro-batch	event_id/hash key 로 중복 방지. Kafka 는 optional
2 Normalize/Split/Gate	batch 와 같은 DataFrame 로직 재사용	batch/stream feature contract 불일치 방지
3 Feature	같은 model_version 으로 partition batch inference	기존 centroid 와 같은 vector space 유지
4 Assign	pain centroid/radius 를 executor 에 broadcast	최근접 pain_id 배정. radius 밖이면 Novel Queue
5 Aggregate	event-time 기준 pain×segment×period count/share 갱신	late event 는 Data Pack 별 allowed_lateness 와 watermark 로 관리 [S4]
6 Sink	foreachBatch 로 PostgreSQL/aggregate table upsert	batch_id 와 unique key 를 이용해 재시도 중복 효과 방지 [S4]
7 Rebuild	Novel Queue 가 support 를 확보하면 offline cluster rebuild	실시간 re-clustering 은 MVP 에서 하지 않음

외부 DB sink 에 대해 “exactly-once”라고 과장하지 않는다. Structured Streaming 의 foreachBatch batchId 를 사용해 processed_batch 원장과 (pipeline_version, pain_id, period) unique key 를 관리하고, 재시도 시 동일 결과를 idempotent upsert 한다. [S4]

8.11 Failure Recovery · Idempotency · Lineage

실패/위험	대응	확인 지표
중복 원천 event	source 별 deterministic event_id 생성 후 dedupe	duplicate_rate / input-output row count
Worker/Task 실패	Spark task retry 와 lineage 기반 stage 재실행	failed task / retry / executor lost
Batch 중간 실패	각 Job output 을 run_id/version path 에 쓰고 전체 검증 후 active_version publish	last_success_run / incomplete run 격리
Streaming restart	checkpoint + processed_batch ledger	restart 후 duplicate aggregate 가 없어야 함
모델 변경	model_version/cluster_version/radius_version 고정	다른 vector space/centroid 혼용 금지
결과 추적	event_id→claim_id→feature_version→pain_id→aggregate→signal	Radar row 에서 원문과 pipeline version 까지 역추적

8.12 Observability — 발표에서 Spark 내부를 수치로 보여준다

Spark UI 와 Event Log/History Server 를 사용해 완료된 Job 도 Stage/Task/Executor 관점에서 다시 확인한다. Spark 공식 Web UI/History 기능은 job, stage, executor, SQL, Structured Streaming 상태와 resource consumption 을 관찰할 수 있다. [S5]

관측 층	기록할 지표	판단
Pipeline	input rows, clean rows, claim rows, candidate ratio, feature rows	데이터 중복·축소가 의도대로 동작하는가

관측 층	기록할 지표	판단
Stage	stage wall time, task count, p50/p95 task time	straggler/skew 가 있는가
Shuffle	shuffle read/write, fetch wait, spill memory/disk	groupBy/join 병목이 network/memory 에 있는가
Executor	active cores, task time, GC time, peak memory	Worker 추가가 실제 compute 병렬성을 만드는가
SQL	scan bytes, partition pruning, broadcast 여부	partition/join 전략이 query plan 에 반영됐는가
Streaming	input rows/sec, processed rows/sec, batch duration, lag/checkpoint	증분 처리량이 입력을 따라가는가

8.13 분산 Benchmark — “분산을 썼다”가 아니라 효과를 비교한다

실험	고정 조건	비교	측정	성공 신호
Scale-Out	동일 claim workload / 동일 code	Worker 1 / 2 / 4	wall time, records/sec, executor-hour, shuffle	Worker 증가에 따른 throughput 과 crossover 를 설명. 항상 선형 증가를 기대하지 않음
Partition	동일 source · 기간 query	비파티션/잘못된 배치 vs source/date partition	scan bytes, task 수, stage time	기간 filter 에서 scan/stage 감소
Join	동일 metadata + fact	shuffle join vs broadcast join	shuffle bytes, join time, memory	작은 metadata broadcast 가 network 비용을 줄임
Incremental	동일 신규 기간 input	전체 재계산 vs 신규 interval 만 처리	reprocessed rows, update time, 결과 일치	재처리 row 와 update time 감소, 결과는 batch 기준과 일치

Benchmark workload 규칙

제품 결과는 실제 CFPB/NHTSA 데이터에서만 산출한다. 분산 인프라 Scale-Out 검증에 더 큰 workload 가 필요하면 실제 Claim 분포를 보존한 deterministic replay/replication 을 별도 benchmark dataset 으로 만들고 “성능 실험용 확장 workload”라고 명시한다. 이를 실제 사용자 의견 수로 주장하지 않는다.

최종 발표에는 최소 ① 1/2/4 Worker 결과표, ② 가장 큰 Shuffle Stage 의 Spark UI/History 캡처, ③ partition 또는 broadcast 적용 전후 한 가지, ④ incremental 재처리 row 수를 함께 보여준다. 이 네 가지가 “분산”의 핵심 증거다.

8.14 기술 스택과 선택 이유

영역	기본 도구	선택 이유
Cluster/Batch	Apache Spark Standalone + PySpark DataFrame	scan/join/groupBy/window/MLlib 을 하나의 engine 에서 처리하고 Worker 수를 통제해 scale-out 실험 가능 [S1]
ML Clustering	Spark MLlib BisectingKMeans	대규모 vector 에 대한 distributed hierarchical split [S3]
Incremental	Spark Structured Streaming + file replay; Kafka optional	batch 와 같은 DataFrame 연산을 재사용하고 micro-batch/checkpoint/watermark 사용 [S4]
Storage	MinIO + Parquet	S3-compatible object lake, columnar scan, partitioned batch output
Embedding	BGE small English 계열 / partition batch inference	Candidate corpus 만 executor 단위 병렬 vectorization. 실패 시 TF-IDF
Serving	PostgreSQL + Spring Boot 또는 FastAPI	Spark 계산과 UX query latency 를 분리
Monitoring	Spark UI + Event Log + History Server	Stage/Task/Executor/SQL/Streaming evidence 저장 [S5]
Deployment	Docker Compose + 2~4 Worker VM/host	재현 가능한 local/VM cluster. Kubernetes/YARN 은 MVP 밖

영역	기본 도구	선택 이유
Orchestration	Python CLI + run manifest / shell workflow	6 주 프로젝트에서는 Airflow 를 추가하지 않고 versioned stage dependency 만 관리

스택 통제

Flink, Hadoop HDFS, Elasticsearch, 별도 Vector DB, graph DB, Kubernetes, Airflow 를 “분산처럼 보이기 위해” 추가하지 않는다. 분산 처리는 Spark Worker · Partition · Shuffle · Scale-Out · Incremental 에서 충분히 증명한다. Kafka 와 Prometheus/Grafana 는 일정이 남을 때만 추가한다.

9. UX / PRODUCT SURFACE

사용자는 모델 구조를 보지 않는다. 화면은 “이 데이터로 답할 수 있는가 → 어떤 Pain 이 왜 지금 중요한가 → 실제 Evidence 가 무엇인가”에 집중한다.

화면	핵심 내용	사용자 판단
1 Ask	자연어 질문 + 예시	무엇을 알고 싶은지 입력
2 Can PainScope Answer?	Pack, FULL/PARTIAL/UNSUPPORTED, 가능/불가능 주장, bias	이 분석을 믿고 시작할지 판단
3 Scope	Product/segment/period/coverage	분석 범위 확정
4 Discovery Radar	Pain label, state, priority, confidence, change, support, spread	지금 조사할 후보 비교
5 Problem Detail	Pain Frame, trend, why-now, representative evidence, lineage	정말 같은 문제인지 원문으로 확인
6 Discovery Brief	발견/근거/한계/추가 조사 질문	조사 과제로 전환

9.1 “Topic → Pain”을 UI 에서도 보여준다

예시

Source Issue: Managing an account

→ Topic: Account closure

→ Validated Pain: “계좌 폐쇄 요청이 반복적으로 처리되지 않아 계좌가 계속 활성화 상태로 남음”

→ Impact Evidence: 일부 complaint 에서 추가 fee 발생

→ State: Growing

사용자는 “Account closure 가 많이 언급됐다”가 아니라 “폐쇄 요청 미처리라는 구체적 실패가 최근 커지고 있다”를 본다.

9.2 Problem Detail 필수 노출

- Context / Failure / Impact / Target 의 Pain Frame.
- 최근 conditional share, baseline, state, support, persistence.
- 대표 Evidence 5~10 개와 원문 event 로 이동 가능한 trace.
- Top-1 company/entity 집중도와 unique entity 수.
- Data Pack limitation: 모집단 denominator 없음, historical corpus cutoff 등.
- “이 결과로 말할 수 없는 것”을 별도 문장으로 고정.

10. DATA STRATEGY — CFPB Main 을 유지하면서 2026 정책 리스크를 통제한다

2026-08-14 CFPB 가 narrative 의 재량 공개를 중단한 것은 PainScope 3.0 의 가장 중요한 외부 리스크다. 그러나 기존에 공개된 narratives 가 사라진 것은 아니며 CFPB 는 previously published narratives 를 public domain 으로 보고 FOIA Reading Room 을 통해 공개한다고 밝혔다. [D2] 프로젝트는 “실시간 금융 트렌드”가 아니라 historical replay 를 명확히 표기한다.

구분	CFPB Main	NHTSA Validation
제품 역할	Pain Discovery 품질 + 대규모 분산 Main Demo	다른 domain/schema portability + fresh text replay
Text freshness	historical narrative corpus cutoff 고정	지속 업데이트 complaint text
Scale	대규모 complaint 기반 main workload	상대적으로 작으므로 scale proof 주장 안 함
Schema	Product/Sub-product/Issue/Sub-issue	Vehicle/Component/Severity
Demo 비중	70~80%	20~30%
주장	당시 새롭게 커지던 Pain 을 발견	같은 Core Engine 이 자동차 complaint 에도 적용

10.1 Week 1 데이터 고정 규칙

1. CFPB historical narrative corpus 의 출처 · cutoff date · row count · narrative coverage 를 고정하고 README 에 기록한다.
2. Credit Card / Checking-Savings / Credit Reporting 중 narrative 품질과 volume 이 안정적인 Primary segment 1 개를 결정한다.
3. CFPB form taxonomy version mapping 을 만들어 historical issue label 변화를 canonical path 로 정리한다.
4. NHTSA 는 1~2 개 component segment 만 선택해 adapter/portability 를 검증한다.
5. 데이터 확보가 불안정하면 기능을 넓히지 말고 primary segment 와 기간을 즉시 축소한다.

11. QUALITY GATES — 비교 실험 없이 “이 결과를 Pain 이라고 불러도 되는가”만 확인한다

3.0 은 일정상 Raw clustering vs Pain Frame, embedding model 비교, K-Means vs BERTopic 같은 실험을 하지 않는다. 대신 아래 Gate 를 통과하지 못한 결과는 제품에서 숨긴다.

Gate	Must-pass 기준	실패 시 조치
Pain Specificity	label 이 “Credit reporting” 같은 대분류가 아니라 Failure 를 포함	segment 축소 또는 cluster 1 회 재분할
Evidence Alignment	대표 Evidence 대부분이 같은 Failure 를 설명	mixed=true → 1 회 재분할, 재실패 시 숨김
Actionability	어느 과정/기능을 조사해야 하는지 Target 지정 가능	Topic Candidate 로 강등
Support	minimum support 이상	Radar 제외, Novel/Watchlist 유지
Evidence Diversity	중복 · 단일 entity 과집중이 아님	Confidence 낮춤 / entity-specific 표기
Traceability	모든 상위 결과가 event_id 원문까지 연결	출시 차단
Answerability	지원 불가 질문을 억지 분석하지 않음	해당 query flow 차단

11.1 최소 QA 샘플

- CFPB Primary segment 의 상위 Pain 5 개는 팀이 대표 Evidence 10~20 개를 직접 읽고 Gate 만 확인한다.
- NHTSA 는 상위 3 개 Pain 이 같은 Pipeline 에서 생성되는지와 Evidence trace 만 확인한다.
- 외부 PM blind review 는 시간이 남으면 수행하지만 성공조건으로 두지 않는다.
- AI 모델 숫자 지표보다 최종 제품에서 잘못된 Topic 을 Pain 이라고 보여주지 않는 안전장치를 우선한다.

12. EXECUTION — 6 주 · 6 인 개발 계획

주차	핵심 작업	완료 기준	Go / No-Go
1 주	CFPB corpus/Pack 고정 + Spark Standalone 2 Worker + MinIO/Parquet smoke test + partition 설계 + NHTSA EDA	Primary segment 1 개, Answerability/Pack 계약, Raw→Clean 분산 Job 과 History log 확보	cluster/스토리지 불안정 시 Kafka·고급 monitoring 보류, batch cluster 부터 안정화
2 주	Clause split, Candidate Gate, partition-batch embedding, segment-aware BisectingKMeans, cluster qualifier	CFPB 에서 Validated Pain 5 개 + claim/event trace + executor 분산 Feature Job 완료	대분류만 나오면 모델 추가보다 segment/gate 수정; embedding 처리량 부족 시 후보 축소/TF-IDF
3 주	Pain model artifact, distributed assignment, Aggregate/Detect, repartition·broadcast·AQE 적용, Spark 병목 관측	Emerging Pain 5 개 + D8 shuffle metrics + partition/join 전략 1 개 이상 수치 확보	skew/spill 과다 시 salting/segment split 후 core batch 고정
4 주	Answerability API, Scope, Radar, Detail, Brief, PostgreSQL serving + active_version publish	CFPB Question→Answerability→Radar→Evidence End-to-End; UI 요청이 Spark full job 없이 동작	신규 화면 금지, serving 과 batch version 연결 안정화
5 주	NHTSA Adapter + Same Engine reuse + incremental file replay; 가능 하면 Kafka/Structured Streaming	NHTSA 동일 Core Job 재사용 + centroid broadcast assignment + idempotent aggregate update	Kafka/stream checkpoint 불안정 시 deterministic file micro-batch 로 즉시 축소
6 주	1/2/4 Worker Scale-Out, partition/broadcast/incremental benchmark, History 캡처, QA/Demo	분산 증거 4 종: scale-out 표, shuffle stage, 최적화 전후 1 개, incremental rows + 최종 제품 demo	미검증 기능은 숨기고 batch+분산 수치가 확실한 범위만 발표

12.1 팀 역할

역할	핵심 책임
PM / Product	Pain 정의, Answerability/Data Pack contract, Quality Gate, demo narrative, 분산 benchmark 의 제품 연결 메시지
AI + UI/UX	Question schema, cluster qualifier, Brief, Ask/Answerability/Radar UX; 대표 Evidence 기반 AI 호출 제한
Data/AI Engineer	Candidate Gate, embedding mapPartitions, BisectingKMeans, centroid/radius, Pain ID/Novel Queue
Data Engineer	CFPB/NHTSA Adapter, Parquet layer, Spark D1~D10, partition/repartition/broadcast/skew, benchmark
Backend/Infra	Spark Standalone Master/Worker, MinIO, PostgreSQL, run/version publish, Structured Streaming/file replay, History Server
Frontend	Ask/Answerability/Radar/Detail/Evidence/Brief + engineering proof 화면/benchmark 결과 노출

12.2 병렬 개발 원칙

- 1~2 주차에는 Frontend 가 실제 데이터 결과를 기다리지 않고 mock contract 로 Answerability/Radar UI 를 병렬 개발한다.
- Data/AI 와 Data Engineering 은 Primary CFPB segment 하나에 집중하고 NHTSA 는 5 주차까지 확장하지 않는다.
- PM 은 매일 “이 결과가 Topic 인가 Pain 인가?”를 대표 Evidence 로 확인하고 명명 규칙을 유지한다.
- 4 주차 이후 신규 모델·신규 source·신규 화면을 추가하지 않는다.

12.3 분산 구현 완료 정의

- 개발용 단일 노드 local mode 만으로 끝내지 않는다. 최종 benchmark 는 Worker 2 개 이상이 참여하는 Spark Standalone 실행을 남긴다.

- Spark UI/History 에서 D4 Feature, D5 Cluster, D8 Aggregate 중 최소 하나의 Stage DAG 와 executor 별 task 분배를 캡처한다.
- 분산 최적화는 최소 2 개를 실제 적용한다: source/date partition pruning, broadcast join, skew 대응, incremental 재처리 중 두 가지 이상.
- 분산 결과의 correctness 를 버리지 않는다. Worker 수나 incremental 방식이 달라도 동일 input/version 에 대한 pain aggregate 결과가 batch 기준과 일치하는지 확인한다.

분산 기능 Cut 기준

Kafka/Prometheus/Grafana 가 없어도 실패가 아니다. 반대로 Spark 를 단일 executor 로만 돌리고 “분산”이라고 발표하면 실패다. 핵심은 2+ Worker 의 실제 task 분배, Shuffle/Partition 최적화, Scale-Out/Incremental 수치다.

13. FINAL DEMO SCENARIO

1. Ask: “최근 신용카드 사용자에게 새롭게 커지는 불편은 무엇인가?”를 입력한다.
2. Answerability: CFPB historical complaint corpus 가 Emerging Pain 질문에는 적합하지만 전체 카드 사용자 prevalence · 원인 확정은 지원하지 않음을 표시한다.
3. Scope: Primary Product 와 기간을 선택한다. 사용자의 분석 요청은 Spark Full Job 을 즉시 실행하는 것이 아니라 검증된 pipeline_version 의 결과 범위를 선택한다.
4. Radar: Popular Topic 이 아니라 Validated Pain Point 를 Discovery Priority 순으로 보여준다.
5. 한 Pain 을 열어 Context/Failure/Impact/Target, conditional share, baseline, Growing/Spike/Persistent 상태를 확인한다.
6. Representative Evidence 를 열어 같은 Failure 가 실제 complaint 원문에 반복되는지와 event→claim→pain lineage 를 확인한다.
7. Discovery Brief 에서 발견 · 근거 · 한계 · 추가 조사 질문만 생성한다.
8. NHTSA 질문으로 전환해 Source Adapter/Data Pack 만 달라지고 동일 Candidate→Feature→Cluster→Aggregate→Detect Core 가 재사용되는 모습을 짧게 보여준다.
9. Engineering Proof 로 전환한다. 동일 Claim workload 를 Worker 1/2/4 에서 실행한 wall time · records/sec · executor-hour 를 보여준다.
10. Spark History 에서 가장 큰 Shuffle Stage(D8 Aggregate)의 task 분배, shuffle read/write, spill/skew 를 보여주고 partition/broadcast 적용 전후 수치 하나를 설명한다.
11. 신규 complaint file/Kafka replay 를 넣어 기존 centroid broadcast assignment 와 aggregate update 가 전체 re-cluster 없이 갱신되는 것을 보여주고, full recompute 대비 재처리 row 수를 비교한다.

최종 발표 메시지

“PainScope 는 Pain 을 찾기 위해 Spark 를 붙인 서비스가 아닙니다. 전체 사용자 의견의 분모를 유지하면서 Claim 을 확장 · vectorize · cluster 하고 Pain × Time 분포를 반복 집계해야 하기 때문에 계산이 분산됩니다. 저희는 같은 결과를 1/2/4 Worker 에서 재현하고 Partition · Shuffle · Incremental 전략이 처리 비용을 어떻게 바꾸는지 함께 증명합니다.”

14. RISK CONTROL & SCOPE CUT

리스크	실패 신호	즉시 대응
Pain 이 Topic 수준에 머물	label 이 Issue/Component 대분류	segment 를 더 좁히고 candidate claim 길이/규칙 수정
Cluster 가 혼합	대표 Evidence 가 서로 다른 Failure	한 번만 bisect; 재실패 시 숨김
CFPB narrative 확보 불안정	cutoff/row/coverage 가 고정되지 않음	historical snapshot 동결, period 축소
CFPB 최신성 공격	2026 정책 이후 fresh narrative 없음	historical replay 명시 + NHTSA fresh validation
LLM hallucination	Evidence 에 없는 원인/해결책 생성	strict JSON + representative evidence only + unsupported_inference reject
Embedding 처리량 부족	2 주차 후보 corpus 처리 불가	Candidate Gate 강화 + TF-IDF fallback
Spark skew/spill	특정 Product/Issue task 장시간 점유	repartition/salting/segment split

리스크	실패 신호	즉시 대응
Streaming 지연	5 주차 checkpoint/retry 불안정	Kafka 제거, deterministic file micro-batch replay
범용 플랫폼 주장 과장	실제 Pack 이 2 개뿐	“확장 단위가 Data Pack”이라고만 주장, all-domain 지원 금지
Worker 수를 늘려도 빨라지지 않음	작은 workload / shuffle · serialization overhead 가 compute 이득보다 큼	실패로 숨기지 않고 scale-out crossover 를 결과로 설명; larger claim workload 에서 재측정
Shuffle/Skew 폭증	D8 특정 task 가 stage 대부분을 점유, spill/fetch wait 증가	repartition key 검토, salting/2-stage aggregate, 대형 segment 별도 Job
Small file 폭증	Parquet object 수 증가, scan task 가 지나치게 많음	physical partition 축소 + batch compaction + target file size 관리
Batch/Serving version 불일치	Radar 는 새 aggregate 인데 Evidence 는 이전 cluster_version	active_version publish 를 원자적 pointer 로 관리하고 API 가 하나의 pipeline_version 만 조회
중분 중복 반영	stream restart 뒤 같은 batch 가 다시 DB 반영	checkpoint + batch_id ledger + unique key idempotent upsert

14.1 MVP 밖

- 세 번째 실제 Data Pack 연결, 외부 실시간 web crawling, 내부 CRM connector.
- 원인 자동 추론, 자동 해결책 추천, Agent 가 개선안을 실행하는 workflow.
- 실시간 re-clustering, 모든 문장 row-level LLM Pain Frame, vector DB 도입.
- NHTSA Recall/Investigation prediction, 회사별 법적 위험 판정.
- 도메인 간 Discovery Priority 절대 비교.
- 모델/알고리즘 비교 실험 및 대규모 hyperparameter tuning.

15. FINAL ACCEPTANCE CRITERIA

축	최소 완료 기준
Product	CFPB Question → Answerability → Scope → Validated Pain Radar → Evidence → Brief 가 동작
Pain Quality	Primary segment 에서 구체적 Validated Pain 5 개 이상; 대분류 Topic 은 Radar 에서 제외
Change	Spike/Growing/Persistent 중 2 종 이상을 실제 Pain 에 표시
Evidence	Top Pain 모두 원문 event 까지 역추적 가능
Answerability	Prevalence/Cause 등 지원 불가 질문은 PARTIAL/UNSUPPORTED 로 처리
Portability	NHTSA 가 Source Adapter/Data Pack 만 다르게 적용되고 Core Pain Pipeline 을 재사용
Distributed	Spark Standalone 2+ Worker 실제 실행, Worker 1/2/4 Scale-Out 수치, Shuffle/Partition 또는 Broadcast 최적화 1 개 이상, Incremental 재처리 감소 증명
Honesty	CFPB historical cutoff, complaint bias, denominator 부재를 UI 와 발표에 명시

15.1 최종 분산 Proof Package

증거	만드시 남길 산출물
Scale-Out Table	동일 workload 1/2/4 Worker 의 wall time, records/sec, executor-hour
Spark History Evidence	D4/D5/D8 중 핵심 Stage 의 DAG, executor/task 분배, shuffle read/write, spill

증거	반드시 남길 산출물
Optimization Evidence	partition pruning 또는 broadcast/skew 대응 적용 전후 수치
Incremental Evidence	full recompute 대비 신규 interval 재처리 rows/update time + 결과 일치
Correctness/Lineage	동일 pipeline_version 의 Radar→Pain→Claim→Event 역추적과 batch/stream aggregate 일치

3.0 의 최종 성공 정의

좋은 화면이나 높은 모델 점수가 아니라, “구체적인 사용자 Failure 를 원문으로 검증할 수 있고, 그 Failure 가 최근 왜 조사 가치가 커졌는지를 설명하며, 질문과 데이터의 한계를 숨기지 않는 것”이 성공이다.

16. REFERENCES & SOURCE NOTES

[D1] CFPB, Consumer Complaint Database — 데이터 구조, 공개 범위, 대표성/volume 해석 주의, taxonomy 변경 이력.

<https://www.consumerfinance.gov/data-research/consumer-complaints/>

[D2] CFPB, “The CFPB to Cease Discretionary Publication of Complaint Narratives and Visualizations”, Aug. 14, 2026 — 신규 narrative 공개 중단 및 기존 공개분의 FOIA/public-domain 처리. <https://www.consumerfinance.gov/about-us/newsroom/the-cfpb-to-cease-discretionary-publication-of-complaint-narratives-and-visualizations/>

[D3] NHTSA Datasets and APIs — Vehicle owner complaints 가 safety issue/defect trend 식별에 사용되며 complaint data 를 공개. <https://www.nhtsa.gov/nhtsa-datasets-and-apis>

[R4] Preoṭiuc-Pietro et al., “Automatically Identifying Complaints in Social Media”, ACL 2019 — complaint 를 expectation breach 관점에서 다룸. <https://aclanthology.org/P19-1495/>

[R5] Lee et al., “Painsight: An Extendable Opinion Mining Framework for Detecting Pain Points Based on Online Customer Reviews”, WASSA 2023. <https://aclanthology.org/2023.wassa-1.20/>

[C1] Brandwatch, Innovation Spotlight / Ask Iris — 자연어 질문, search 생성, follow-up 분석. <https://www.brandwatch.com/product-updates/product-innovation/>

[C2] Qualtrics XM Discover, AI-assisted / Automated Text Analytics — AI topic hierarchy 와 unstructured feedback 분석. <https://www.qualtrics.com/support/xm-discover/text-analytics/ai-assisted-text-analytics-in-xm-discover/>

[C3] Quid, Product Release: Dataset Library — dataset selector + Ask Q grounded answer, Jul. 30, 2026. <https://www.quid.com/knowledge-hub/resource-library/blog/product-release-dataset-library>

[C4] Thematic — bottom-up themes, emerging issue, exact-word traceability 와 prioritization. <https://getthematic.com/>

[U1] G2, Brandwatch Consumer Intelligence Reviews 2026 — 사용자 경험/학습비용 등 참고. <https://www.g2.com/products/brandwatch-consumer-intelligence/reviews>

[U2] G2, Qualtrics Customer Experience Reviews — advanced feature learning curve 관련 사용자 피드백. <https://www.g2.com/products/qualtrics-customer-experience/reviews>

[U3] G2, Quid Reviews 2026 — 신규 사용자 learning curve 관련 사용자 피드백. <https://www.g2.com/products/quid-2023-10-05/reviews>

[S1] Apache Spark, Standalone Mode — Master/Worker cluster 구성과 standalone deploy mode. <https://spark.apache.org/docs/latest/spark-standalone.html>

[S2] Apache Spark, SQL Performance Tuning — Adaptive Query Execution, partition coalescing/skew optimization 등. <https://spark.apache.org/docs/latest/sql-performance-tuning.html>

[S3] Apache Spark MLlib, BisectingKMeans — 같은 level 의 bisect step 을 묶어 parallelism 을 높이는 distributed clustering 구현. <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.clustering.BisectingKMeans.html>

[S4] Apache Spark Structured Streaming — foreachBatch, watermark, micro-batch/checkpoint 관련 공식 문서. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

[S5] Apache Spark, Monitoring and Web UI — Spark UI/Event Log/History Server 를 통한 Job · Stage · Executor · SQL · Streaming 관측. <https://spark.apache.org/docs/latest/monitoring.html>

Source note: 경쟁사의 “구조적 한계”는 공식 기능 설명과 공개 사용자 리뷰를 바탕으로 PainScope 와의 제품 경계를 해석한 기획 판단이다. 사용자 리뷰는 표본 편향이 있을 수 있으므로 단일 리뷰를 일반화하지 않는다. CFPB/NHTSA 의 데이터 해석 한계는 각 기관의 공식 설명을 우선한다.

PainScope 3.0

Topic → Trend 가 아니라

Pain Claim → Validated Pain → Change → Evidence

CFPB proves value. NHTSA proves portability.